StudiesTalk App Summary

Repo-backed one-page overview generated from code, package metadata, and docs in this workspace.

What It Is

StudiesTalk is a full-stack, multi-tenant workspace app for language schools. Repo evidence shows chat, class operations, analytics, email, AI voice practice, and admin tooling in one web application.

Who It's For

Primary persona: a language school administrator or teacher running a school workspace; student-facing flows also exist in the repo.

What It Does

- Multi-tenant workspaces with role handling for **student**, **teacher**, **admin**, and **super_admin**.
- Channels, direct messages, threaded replies, reactions, typing indicators, search, and file sharing.
- Tasks, comments, homework/class planning hooks, announcements, calendar events, and attendance endpoints.
- Live classes with Jitsi token issuance, room scheduling, session join flow, and slide state streaming/sync.
- Analytics dashboards with school, teacher, and student overview endpoints.
- AI features including chat, streaming chat, realtime voice practice, runtime tracking, and workspace budget controls.
- Workspace email settings, templates, inbox/reply flows, plus OTP, password reset, and admin security controls.

How It Works

Frontend: Static assets in *public/* provide the main shell (*index.html*, *app.js*) and feature modules for analytics, calendar, announcements, live classes, and AI voice practice.

Backend: *server.js* runs an Express app with JSON/form parsing, static hosting, cookies, Helmet, CORS, rate limits, CSRF checks, uploads, and many REST endpoints; *server/routes/live.js* and *server/routes/liveSchedule.js* mount live-class flows.

Data + services: SQLite via *better-sqlite3* defaults to *worknest.db*; uploads live in *uploads/*; email attachments live in *storage/email_attachments/*. Optional integrations in code include Jitsi, OpenAI realtime, Google Translate, Twilio OTP, and SMTP/IMAP email.

Data flow: Browser UI -> Express REST endpoints -> SQLite / local file storage -> optional external services when configured.

How To Run

- Install dependencies: **npm install**
- Create a **.env** file. Exact example: **Not found in repo**. Minimum local defaults visible in code include **PORT** (3000 by default), JWT secrets, DB/upload paths, and optional service keys.
- Start the app: **npm start** or **npm run dev**
- Open **http://localhost:3000** for the main app; **/admin** is also served.

Notes: Database migrations/setup instructions for a full production environment are partial across repo docs; a single authoritative local setup guide was not found.